

Estrutura de dados

Aluno: Bruno Victor Paiva da Silva

1. Implemente as estruturas de dados: 1) array ordenado, 2) lista encadeada não ordenada, e 3) árvore de busca binária balanceada (e.g., AVL, Red-Black trees). Em seguida, insira n elementos em cada estrutura de dados. Depois disso, considerando diferentes valores de n , compare os tempos de inserção e busca de cada uma.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>

class SortedArray {
private:
    std::vector<int> arr;
public:
    void insert(int val){
        arr.push_back(val);
        int j = arr.size() - 2;

        while(j >= 0 && arr[j] > val){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = val;
    }

    bool search(int val){
        int esq = 0;
        int dir = arr.size() - 1;
```

```

        while(esq <= dir){
            int meio = esq + (dir - esq) / 2;

            if(arr[meio] == val){
                return true;
            }

            if(arr[meio] < val) {
                esq = meio + 1;
            } else {
                dir = meio - 1;
            }
        }

        return false;
    }
};

struct NodeLista{
    int data;
    NodeLista* next;

    NodeLista(int val) : data(val), next(nullptr) {}
};

class LinkedList {
private:
    NodeLista* head;

public:
    LinkedList() : head(nullptr) {}

    void insert(int val){
        NodeLista* newNode = new NodeLista(val);
        newNode->next = head;
        head = newNode;
    }
};

```

```

    }

    bool search(int val){
        NodeLista* curr = head;

        while(curr != nullptr){
            if(curr->data == val){
                return true;
            }

            curr = curr->next;
        }
        return false;
    }
};

struct NodeAVL {
    int data, height;
    NodeAVL *left, *right;
    NodeAVL(int val) : data(val), height(1), left(nullptr), right(nullptr) {}
};

class AVLTree {
private:
    NodeAVL* root;

    int height(NodeAVL* n) { return n ? n->height : 0; }
    int getBalance(NodeAVL* n) { return n ? height(n->left) - height(n->right) : 0; }

    NodeAVL* rightRotate(NodeAVL* y){
        NodeAVL* x = y->left;
        NodeAVL* T2 = x->right;
        x->right = y;
        y->left = T2;
    }
};

```

```

        y->height = std::max(height(y->left), height(y->right)) + 1;
        x->height = std::max(height(x->left), height(x->right)) + 1;
        return x;
    }

    NodeAVL* leftRotate(NodeAVL* x){
        NodeAVL* y = x->right;
        NodeAVL* T2 = y->left;
        y->left = x;
        x->right = T2;
        x->height = std::max(height(x->left), height(x->right)) + 1;
        y->height = std::max(height(y->left), height(y->right)) + 1;
        return y;
    }

    NodeAVL* insertNode(NodeAVL* node, int val) {
        if(!node) return new NodeAVL(val);
        if(val < node->data) node->left = insertNode(node->left, val);
        else if(val > node->data) node->right = insertNode(node->right, val);
        else return node;

        node->height = 1 + std::max(height(node->left), height(node->right));
        int balance = getBalance(node);

        if(balance > 1 && val < node->left->data) return rightRotate(node);
        if(balance < -1 && val > node->right->data) return leftRotate(node);
        if(balance > 1 && val > node->left->data) {

```

```

        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if(balance < -1 && val < node->right->data){
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

bool searchNode(NodeAVL* node, int val) {
    if(!node) return false;
    if(node->data == val) return true;
    if(val < node->data) return searchNode(node->left, va
l);

    return searchNode(node->right, val);
}

public:
    AVLTree() : root(nullptr){}
    void insert(int val) {root = insertNode(root, val);}
    bool search(int val) {return searchNode(root, val);}
};

int main(){

    std::vector<int> tamanhos = {1000, 5000, 10000, 20000, 50
000, 100000, 200000};
    std::mt19937 rng(42);

    std::cout << "N | Estrutura | Inserção | Busca" << std::e
ndl;

    for(int n : tamanhos){
        std::vector<int> dados(n);
        for(int i = 0; i < n; i++) dados[i] = i;
        shuffle(dados.begin(), dados.end(), rng);
    }
}

```

```

LinkedList lista;
SortedArray arr;
AVLTree avl;

auto t1 = std::chrono::high_resolution_clock::now();
for(int x : dados) lista.insert(x);
auto t2 = std::chrono::high_resolution_clock::now();
auto d_ins_lista = std::chrono::duration<double>(t2 -
t1).count();

t1 = std::chrono::high_resolution_clock::now();
for(int i = 0; i < 1000; i++) lista.search(rng() %
n);
t2 = std::chrono::high_resolution_clock::now();
auto d_bus_lista = std::chrono::duration<double>(t2 -
t1).count();

t1 = std::chrono::high_resolution_clock::now();
for(int x : dados) arr.insert(x);
t2 = std::chrono::high_resolution_clock::now();
auto d_ins_arr = std::chrono::duration<double>(t2 - t
1).count();

t1 = std::chrono::high_resolution_clock::now();
for(int i = 0; i < 1000; i++) arr.search(rng() % n);
t2 = std::chrono::high_resolution_clock::now();
auto d_bus_arr = std::chrono::duration<double>(t2 - t
1).count();

t1 = std::chrono::high_resolution_clock::now();
for(int x : dados) avl.insert(x);
t2 = std::chrono::high_resolution_clock::now();
auto d_ins_avl = std::chrono::duration<double>(t2 - t
1).count();

```

```

    t1 = std::chrono::high_resolution_clock::now();
    for(int i = 0; i < 1000; i++) avl.search(rng() % n);
    t2 = std::chrono::high_resolution_clock::now();
    auto d_bus_avl = std::chrono::duration<double>(t2 - t
1).count();

    std::cout << std::fixed;
    std::cout << n << " | Lista | " << d_ins_lista << " |
" << d_bus_lista << std::endl;
    std::cout << n << " | Array | " << d_ins_arr << " | "
<< d_bus_arr << std::endl;
    std::cout << n << " | AVL | " << d_ins_avl << " | "
<< d_bus_avl << std::endl;
    }

    return 0;
}

```

```
rm -f estruturas
g++ -Wall -Wextra -O3 -std=c++17 -o estruturas estruturas.cpp
N | Estrutura | Inserção | Busca
1000 | Lista | 0.000055 | 0.000011
1000 | Array | 0.000287 | 0.000011
1000 | AVL | 0.000186 | 0.000042
5000 | Lista | 0.000270 | 0.000010
5000 | Array | 0.004252 | 0.000007
5000 | AVL | 0.001269 | 0.000072
10000 | Lista | 0.000455 | 0.000009
10000 | Array | 0.016977 | 0.000006
10000 | AVL | 0.001805 | 0.000057
20000 | Lista | 0.000868 | 0.000011
20000 | Array | 0.066904 | 0.000006
20000 | AVL | 0.004946 | 0.000104
50000 | Lista | 0.002658 | 0.000007
50000 | Array | 0.384909 | 0.000011
50000 | AVL | 0.016337 | 0.000188
100000 | Lista | 0.004127 | 0.000006
100000 | Array | 1.595690 | 0.000009
100000 | AVL | 0.027198 | 0.000171
200000 | Lista | 0.007013 | 0.000010
200000 | Array | 5.704295 | 0.000007
200000 | AVL | 0.109535 | 0.000296

Execução finalizada!
bruno@pop-os:~/programacao-paralela/algoritmos-avancados/estrutura/estruturas$
```

2. Implemente tabelas hash com encadeamento e usando endereçamento aberto. Realize experimentos para mostrar numericamente as vantagens e desvantagens de cada caso, considerando as operações de inserção, busca e remoção.

```
#include <iostream>
#include <vector>
#include <list>
#include <chrono>
#include <random>
#include <iomanip>
#include <algorithm>

class ChainingHash {
private:
    std::vector<std::list<int>> table;
```



```

    int numElements;

    int hashFunction(int key) {
        return key % table.size();
    }

public:
    ChainingHash(int size) : table(size), numElements(0) {}

    void insert(int key) {
        int index = hashFunction(key);
        for (int k : table[index]) {
            if (k == key) return;
        }
        table[index].push_front(key);
        numElements++;
    }

    bool search(int key) {
        int index = hashFunction(key);
        for (int k : table[index]) {
            if (k == key) return true;
        }
        return false;
    }

    void remove(int key) {
        int index = hashFunction(key);
        table[index].remove(key);
    }
};

class OpenAddressingHash {
private:
    enum State { EMPTY, OCCUPIED, DELETED };

```

```

std::vector<int> table;
std::vector<State> state;
int capacity;
int numElements;

int hashFunction(int key) {
    return key % capacity;
}

public:
    OpenAddressingHash(int size) : table(size), state(size, EMPTY), capacity(size), numElements(0) {}

    void insert(int key) {
        if (numElements == capacity) return;

        int index = hashFunction(key);

        while (state[index] == OCCUPIED) {
            if (table[index] == key) return;
            index = (index + 1) % capacity;
        }

        table[index] = key;
        state[index] = OCCUPIED;
        numElements++;
    }

    bool search(int key) {
        int index = hashFunction(key);
        int startIndex = index;

        while (state[index] != EMPTY) {
            if (state[index] == OCCUPIED && table[index] == key) {
                return true;
            }
            index = (index + 1) % capacity;
        }
    }

```

```

        }
        index = (index + 1) % capacity;
        if (index == startIndex) break;
    }
    return false;
}

void remove(int key) {
    int index = hashFunction(key);
    int startIndex = index;

    while (state[index] != EMPTY) {
        if (state[index] == OCCUPIED && table[index] == k
ey) {
            state[index] = DELETED;
            numElements--;
            return;
        }
        index = (index + 1) % capacity;
        if (index == startIndex) break;
    }
}

};

int main() {
    int TABLE_SIZE = 100000;
    std::vector<int> tamanhos = {50000, 80000, 95000, 99000};
    std::mt19937 rng(42);

    std::cout << std::fixed << std::setprecision(6);
    std::cout << "Tam_Tabela = " << TABLE_SIZE << "\n";
    std::cout << "N | Estrutura | Insercao (s) | Busca (s) |
Remocao (s)\n";

    for (int n : tamanhos) {
        std::vector<int> dados(n);

```

```

for(int i = 0; i < n; i++) dados[i] = i * 7;
std::shuffle(dados.begin(), dados.end(), rng);

std::vector<int> alvos_teste(10000);
for(int i = 0; i < 10000; i++) alvos_teste[i] = dados
[rng() % n];

ChainingHash chainHash(TABLE_SIZE);
OpenAddressingHash openHash(TABLE_SIZE);

auto t1 = std::chrono::high_resolution_clock::now();
for(int x : dados) chainHash.insert(x);
auto t2 = std::chrono::high_resolution_clock::now();
auto d_ins_chain = std::chrono::duration<double>(t2 -
t1).count();

t1 = std::chrono::high_resolution_clock::now();
for(int x : alvos_teste) chainHash.search(x);
t2 = std::chrono::high_resolution_clock::now();
auto d_bus_chain = std::chrono::duration<double>(t2 -
t1).count();

t1 = std::chrono::high_resolution_clock::now();
for(int x : alvos_teste) chainHash.remove(x);
t2 = std::chrono::high_resolution_clock::now();
auto d_rem_chain = std::chrono::duration<double>(t2 -
t1).count();

t1 = std::chrono::high_resolution_clock::now();
for(int x : dados) openHash.insert(x);
t2 = std::chrono::high_resolution_clock::now();
auto d_ins_open = std::chrono::duration<double>(t2 -
t1).count();

t1 = std::chrono::high_resolution_clock::now();
for(int x : alvos_teste) openHash.search(x);

```

```

        t2 = std::chrono::high_resolution_clock::now();
        auto d_bus_open = std::chrono::duration<double>(t2 -
t1).count();

        t1 = std::chrono::high_resolution_clock::now();
        for(int x : alvos_teste) openHash.remove(x);
        t2 = std::chrono::high_resolution_clock::now();
        auto d_rem_open = std::chrono::duration<double>(t2 -
t1).count();

        std::cout << n << " | Chaining | " << d_ins_chain <<
" | " << d_bus_chain << " | " << d_rem_chain << "\n";
        std::cout << n << " | Open Addr | " << d_ins_open <<
" | " << d_bus_open << " | " << d_rem_open << "\n";
    }

    return 0;
}

```

```

rm -f benchmark_hash
g++ -Wall -Wextra -O3 -std=c++17 -o benchmark_hash main.cpp

```

```

Tam_Tabela = 100000
N | Estrutura | Insercao (s) | Busca (s) | Remocao (s)
50000 | Chaining | 0.006128 | 0.000001 | 0.001214
50000 | Open Addr | 0.000321 | 0.000001 | 0.000043
80000 | Chaining | 0.009888 | 0.000001 | 0.002155
80000 | Open Addr | 0.000606 | 0.000001 | 0.000064
95000 | Chaining | 0.007319 | 0.000002 | 0.001790
95000 | Open Addr | 0.000595 | 0.000001 | 0.045072
99000 | Chaining | 0.006332 | 0.000001 | 0.001150
99000 | Open Addr | 0.000428 | 0.000003 | 0.088002

```

```

❖ bruno@pop-os: ~/programacao-paralela/algoritmos-avancados/estrutura/hash$

```

3. Compare implementações de tabelas hash com hashing duplo e probing linear. Analise e justifique os resultados dos seus experimentos.

```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <iomanip>
#include <algorithm>

bool isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i * i <= n; i = i + 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}

int nextPrime(int n) {
    int prime = n;
    while (true) {
        prime++;
        if (isPrime(prime)) return prime;
    }
}

int prevPrime(int n) {
    int prime = n;
    while (true) {
        prime--;
        if (isPrime(prime)) return prime;
    }
}

class LinearProbingHash {
private:

```

```

enum State { EMPTY, OCCUPIED, DELETED };
std::vector<int> table;
std::vector<State> state;
int capacity;
int numElements;

int hash1(int key) { return key % capacity; }

void rehash() {
    int oldCapacity = capacity;
    capacity = capacity * 2;

    std::vector<int> oldTable = table;
    std::vector<State> oldState = state;

    table.assign(capacity, 0);
    state.assign(capacity, EMPTY);
    numElements = 0;

    for (int i = 0; i < oldCapacity; i++) {
        if (oldState[i] == OCCUPIED) {
            insert(oldTable[i]);
        }
    }
}

public:
    LinearProbingHash(int size) : table(size), state(size, EM
PTY), capacity(size), numElements(0) {}

    void insert(int key) {
        if ((float)numElements / capacity >= 0.75) {
            rehash();
        }

        int index = hash1(key);

```

```

        while (state[index] == OCCUPIED) {
            if (table[index] == key) return;
            index = (index + 1) % capacity;
        }

        table[index] = key;
        state[index] = OCCUPIED;
        numElements++;
    }

    bool search(int key) {
        int index = hash1(key);
        int startIndex = index;

        while (state[index] != EMPTY) {
            if (state[index] == OCCUPIED && table[index] == k
ey) return true;
            index = (index + 1) % capacity;
            if (index == startIndex) break;
        }
        return false;
    }
};

class DoubleHashingHash {
private:
    enum State { EMPTY, OCCUPIED, DELETED };
    std::vector<int> table;
    std::vector<State> state;
    int capacity;
    int numElements;
    int prime;

    int hash1(int key) { return key % capacity; }
    int hash2(int key) { return prime - (key % prime); }

```



```

void rehash() {
    int oldCapacity = capacity;
    capacity = nextPrime(capacity * 2);
    prime = prevPrime(capacity);

    std::vector<int> oldTable = table;
    std::vector<State> oldState = state;

    table.assign(capacity, 0);
    state.assign(capacity, EMPTY);
    numElements = 0;

    for (int i = 0; i < oldCapacity; i++) {
        if (oldState[i] == OCCUPIED) {
            insert(oldTable[i]);
        }
    }
}

public:
    DoubleHashingHash(int size) : table(size), state(size, EM
PTY), capacity(size), numElements(0) {
        prime = prevPrime(capacity);
    }

    void insert(int key) {
        if ((float)numElements / capacity >= 0.75) {
            rehash();
        }

        int index = hash1(key);

        if (state[index] == OCCUPIED) {
            int step = hash2(key);

```

```

        while (state[index] == OCCUPIED) {
            if (table[index] == key) return;
            index = (index + step) % capacity;
        }
    }

    table[index] = key;
    state[index] = OCCUPIED;
    numElements++;
}

bool search(int key) {
    int index = hash1(key);

    if (state[index] != EMPTY && (state[index] == DELETED
|| table[index] != key)) {
        int step = hash2(key);
        int startIndex = index;

        while (state[index] != EMPTY) {
            if (state[index] == OCCUPIED && table[index]
== key) return true;
            index = (index + step) % capacity;
            if (index == startIndex) break;
        }
        } else if (state[index] == OCCUPIED && table[index] =
= key) {
            return true;
        }

        return false;
    }
};

int main() {
    int TABLE_SIZE = 100003;

```

```

std::vector<int> tamanhos = {1000000, 5000000, 10000000};
std::mt19937 rng(42);

std::cout << std::fixed << std::setprecision(6);
std::cout << "Tam_Tabela Inicial = " << TABLE_SIZE << "
(Primo)\n";
std::cout << "N          | Estrutura      | Insercao (s) | Bu
sca Erro (s)\n";

for (int n : tamanhos) {
    std::vector<int> dados(n);
    for(int i = 0; i < n; i++) dados[i] = i * 13;
    std::shuffle(dados.begin(), dados.end(), rng);

    std::vector<int> alvos_erro(10000);
    for(int i = 0; i < 10000; i++) alvos_erro[i] = (i * 1
7) + 1;

    LinearProbingHash linearHash(TABLE_SIZE);
    DoubleHashingHash doubleHash(TABLE_SIZE);

    auto t1 = std::chrono::high_resolution_clock::now();
    for(int x : dados) linearHash.insert(x);
    auto t2 = std::chrono::high_resolution_clock::now();
    auto d_ins_lin = std::chrono::duration<double>(t2 - t
1).count();

    t1 = std::chrono::high_resolution_clock::now();
    volatile int enc_lin = 0;
    for(int x : alvos_erro) if (linearHash.search(x)) enc
_lin++;
    t2 = std::chrono::high_resolution_clock::now();
    auto d_bus_lin = std::chrono::duration<double>(t2 - t
1).count();

```

```

    t1 = std::chrono::high_resolution_clock::now();
    for(int x : dados) doubleHash.insert(x);
    t2 = std::chrono::high_resolution_clock::now();
    auto d_ins_doub = std::chrono::duration<double>(t2 -
t1).count();

    t1 = std::chrono::high_resolution_clock::now();
    volatile int enc_doub = 0;
    for(int x : alvos_erro) if (doubleHash.search(x)) enc
_doub++;
    t2 = std::chrono::high_resolution_clock::now();
    auto d_bus_doub = std::chrono::duration<double>(t2 -
t1).count();

    std::cout << n << " | Sond. Linear | " << d_ins_lin
<< "
    | " << d_bus_lin << "\n";
    std::cout << n << " | Hash. Duplo | " << d_ins_doub
<< "
    | " << d_bus_doub << "\n";
}

return 0;
}

```

```

rm -f benchmark_hash
g++ -Wall -Wextra -O3 -std=c++17 -o benchmark_hash main.cpp

```

Tam_Tabela Inicial = 100003 (Primo)

N	Estrutura	Insercao (s)	Busca Erro (s)
1000000	Sond. Linear	0.071061	0.000153
1000000	Hash. Duplo	0.099783	0.000142
5000000	Sond. Linear	0.500446	0.000181
5000000	Hash. Duplo	0.549136	0.000192
10000000	Sond. Linear	1.035386	0.000164
10000000	Hash. Duplo	1.188886	0.026478

Operação finalizado!

❖ bruno@pop-os:~/programacao-paralela/algoritmos-avancados/estrutura/hash2\$